Objetivos da aula:

- Compreender o papel dos padrões estruturais no design de sistemas orientados a objetos.
- Identificar problemas de organização e integração entre classes em aplicações de software.
- Aplicar os padrões Facade, Adapter e Decorator utilizando TypeScript.
- Reconhecer situações práticas em que esses padrões podem ser utilizados em aplicações Node.js e React.
- Desenvolver soluções que reduzam acoplamento, melhorem a organização do código e facilitem a evolução do sistema.

Índice:

i. Introdução aos Padrões Estruturais
ii. Factory Pattern
iii. Adapter Pattern
iv. Decorator Pattern

i. Introdução aos Padrões Estruturais

1.1. Introdução

Na aula anterior foram estudados os padrões criacionais, que tratam da forma como os objetos são criados em um sistema.

No entanto, em aplicações maiores, não basta apenas criar objetos de forma adequada. Também é necessário organizar esses objetos de maneira que o sistema seja:

- fácil de manter
- flexível para evoluir
- simples de compreender

Para resolver esse tipo de problema, utilizam-se os padrões estruturais.

1.2. O que são Padrões Estruturais

Os padrões estruturais são padrões de projeto que tratam da **forma como classes e objetos são organizados e combinados para formar estruturas maiores**.

Esses padrões ajudam a definir **como diferentes partes do sistema podem trabalhar juntas**, mantendo o código organizado e com baixo acoplamento.

Em outras palavras, os padrões estruturais ajudam a responder perguntas como:

- Como conectar diferentes partes do sistema?
- Como simplificar o acesso a sistemas complexos?
- Como adaptar classes incompatíveis?
- Como estender funcionalidades sem modificar o código existente?

1.3. Padrões Criacionais

A Gang of Four definiu **sete padrões estruturais**:

1. Adapter: adaptar interfaces incompatíveis
2. Bridge: separar abstração da implementação
3. Composite: tratar objetos e grupos de forma uniforme
4. Decorator: adicionar funcionalidades a objetos dinamicamente
5. Facade: simplificar o acesso a sistemas complexos
6. Flyweight: reduzir consumo de memória compartilhando objetos
7. Proxy: controlar acesso a um objeto

Nesta aula serão explorados alguns desses padrões que aparecem com mais frequência em aplicações modernas.

ii. Facade Pattern

2.1. Introdução

Em aplicações reais, muitas funcionalidades dependem da interação entre **vários serviços diferentes**. À medida que o sistema cresce, essas interações podem tornar o código mais complexo e difícil de manter.

Considere, por exemplo, o processo de **finalizar uma compra em um sistema de e-commerce**. Essa operação pode envolver várias etapas:

- validar o usuário
- verificar estoque
- criar o pedido
- processar o pagamento
- enviar confirmação por e-mail

Cada uma dessas tarefas pode estar implementada em classes diferentes.

```
class UsuarioService {}
class EstoqueService {}
class PedidoService {}
class PagamentoService {}
class EmailService {}
```

Quando diferentes partes do sistema precisam executar esse processo, elas passam a depender de várias classes ao mesmo tempo.

Esse tipo de situação pode gerar **alto acoplamento entre as partes do sistema**.

Para resolver esse problema, podemos utilizar o Facade Pattern.

2.2. O que é o Facade Pattern

O Facade Pattern é um padrão estrutural que fornece **uma interface simplificada para um conjunto de classes ou subsistemas complexos**.

Em vez de o cliente interagir diretamente com várias classes, ele utiliza **uma única classe que encapsula toda a complexidade do subsistema**.

Essa classe é chamada de Facade.

O objetivo principal desse padrão é:

- simplificar o uso de sistemas complexos
- reduzir o número de dependências entre classes
- melhorar a organização da aplicação

2.3. Problema sem Facade

Considere uma classe responsável por finalizar uma compra.

```
class CheckoutController {
  finalizarCompra() {
    const usuarioService = new UsuarioService();
    const estoqueService = new EstoqueService();
    const pedidoService = new PedidoService();
    const pagamentoService = new PagamentoService();
    const emailService = new EmailService();

    usuarioService.validarUsuario();
    estoqueService.verificarEstoque();
    pedidoService.criarPedido();
    pagamentoService.processarPagamento();
    emailService.enviarConfirmacao();
  }
}
```

Nesse caso, a classe `CheckoutController` precisa conhecer todas as classes do sistema.

Isso gera alguns problemas:

- alto acoplamento entre classes
- dificuldade de manutenção
- repetição de lógica em diferentes partes do sistema

Se outra parte da aplicação também precisar finalizar uma compra, ela terá que repetir toda essa lógica.

2.4. Aplicando o Facade Pattern

Podemos criar uma classe que **centraliza todo esse processo**.

```
class CheckoutFacade {
  private usuarioService = new UsuarioService();
  private estoqueService = new EstoqueService();
  private pedidoService = new PedidoService();
```

```
    private pagamentoService = new PagamentoService();
    private emailService = new EmailService();

    finalizarCompra() {
      this.usuarioService.validarUsuario();
      this.estoqueService.verificarEstoque();
      this.pedidoService.criarPedido();
      this.pagamentoService.processarPagamento();
      this.emailService.enviarConfirmacao();
    }
  }
```

Agora o código que utiliza o sistema fica muito mais simples.

```
  class CheckoutController {
    finalizarCompra() {
      const checkout = new CheckoutFacade();

      checkout.finalizarCompra();
    }
  }
```

2.5. O que realmente mudou

Sem Facade, o cliente precisa conhecer várias classes:

```
CheckoutController (cliente)
      |
  ┌───┼──────────────────────────┐
UsuarioService
EstoqueService
PedidoService
PagamentoService
EmailService
```

Com Facade, o cliente depende apenas de uma classe:

```
CheckoutController (cliente)
          |
    CheckoutFacade
          |
  ┌───────┼──────────────────────────┐
UsuarioService
EstoqueService
PedidoService
PagamentoService
EmailService
```

Ou seja, o cliente **não precisa mais conhecer os detalhes do subsistema**.

2.6. Vantagem prática

Imagine que agora o sistema precisa adicionar uma nova etapa no processo de compra:

- registrar a compra no sistema de auditoria

Sem Facade, seria necessário alterar **todas as partes do sistema que executam o checkout**.

Com Facade, basta alterar apenas a classe `CheckoutFacade`.

```
  class CheckoutFacade {
```

```
    private usuarioService = new UsuarioService();
    private estoqueService = new EstoqueService();
    private pedidoService = new PedidoService();
    private pagamentoService = new PagamentoService();
    private emailService = new EmailService();

    finalizarCompra() {
      this.usuarioService.validarUsuario();
      this.estoqueService.verificarEstoque();
      this.pedidoService.criarPedido();
      this.pagamentoService.processarPagamento();
      this.emailService.enviarConfirmacao();

      console.log("Registrando operação no sistema de auditoria");
    }
  }
```

Nenhuma outra classe precisa ser modificada.

No React hooks frequentemente funcionam como uma fachada para várias operações:

```
const { login } = useAuth();
```

Internamente, esse hook pode utilizar vários serviços diferentes.

2.7. Conclusão

O Facade Pattern é utilizado quando um sistema possui **vários componentes interagindo entre si**, e desejamos fornecer **uma interface simples para utilizá-los**.

Esse padrão traz benefícios importantes:

- simplifica o uso de subsistemas complexos
- reduz o acoplamento entre classes
- centraliza operações complexas em um único ponto

iii. Adapter Pattern

3.1. Introdução

Em sistemas reais, é comum precisar integrar **componentes que foram desenvolvidos de forma independente**. Muitas vezes esses componentes possuem **interfaces diferentes**, o que dificulta sua utilização direta.

Por exemplo, imagine que uma aplicação foi desenvolvida para utilizar um serviço de pagamento específico. Posteriormente, surge a necessidade de integrar um novo provedor de pagamento cuja interface é diferente da esperada pelo sistema.

Nessa situação, alterar todo o código existente para adaptar à nova interface pode ser trabalhoso e arriscado.

Para resolver esse tipo de problema, podemos utilizar o Adapter Pattern.

3.2. O que é o Adapter Pattern

O Adapter Pattern é um padrão estrutural utilizado para **converter a interface de uma classe em outra interface esperada pelo cliente**.

Em outras palavras, o Adapter permite que **classes com interfaces incompatíveis possam trabalhar juntas**.

Ele funciona como um **adaptador**, semelhante aos adaptadores utilizados em tomadas elétricas.

Por exemplo:

`Tomada americana → Adaptador → Tomada brasileira`

No software, a ideia é semelhante: o Adapter permite conectar **interfaces incompatíveis sem modificar o código original**.

3.3. Problema sem Adapter

Considere um sistema que utiliza um serviço de pagamento com a seguinte interface:

```
class PaymentService {
  pay(valor: number) {
    console.log("Pagamento realizado:", valor);
  }
}
```

Agora imagine que precisamos integrar uma **biblioteca externa** que possui uma interface diferente:

```
class ExternalPaymentAPI {
  makePayment(amount: number) {
    console.log("Pagamento externo realizado:", amount);
  }
}
```

Observe que os métodos possuem nomes diferentes:

Sistema	Método
`PaymentService`	`pay()`
`ExternalPaymentAPI`	`makePayment()`

Se tentarmos utilizar diretamente a API externa, o sistema não funcionará corretamente.

3.4. Aplicando o Adapter Pattern

Podemos criar uma classe que **adapte a interface da API externa para o formato esperado pelo sistema**.

```
class PaymentAdapter {
  private externalAPI = new ExternalPaymentAPI();

  pay(valor: number) {
    this.externalAPI.makePayment(valor);
  }
}
```

Agora o sistema pode utilizar o adaptador sem precisar modificar o restante do código.

```
const pagamento = new PaymentAdapter();
pagamento.pay(100);
```

Nesse caso, o Adapter **traduz a chamada do sistema para a interface da API externa**.

Fluxo de funcionamento:

```
Cliente → Adapter → Classe existente
```

3.5. Benefícios do Adapter Pattern

O uso do Adapter traz várias vantagens:

- permite integrar classes com interfaces incompatíveis
- evita modificar código existente
- facilita a integração com bibliotecas externas
- melhora a reutilização de código

Esse padrão é muito comum em situações como:

- integração com APIs externas
- adaptação de bibliotecas de terceiros
- migração entre sistemas antigos e novos

iv. Decorator Pattern

4.1. Introdução

Em muitos sistemas, pode surgir a necessidade de **adicionar funcionalidades extras a um objeto**.

Uma solução comum seria modificar diretamente a classe original. No entanto, essa abordagem pode gerar alguns problemas:

- alteração de código já existente
- aumento da complexidade da classe
- dificuldade de reutilização

Considere, por exemplo, um sistema de envio de notificações.

```
class Notificacao {
  enviar(mensagem: string) {
    console.log("Mensagem enviada:", mensagem);
  }
}
```

Suponha que agora o sistema precise adicionar funcionalidades extras, como:

- registrar logs
- enviar notificações por múltiplos canais

- aplicar criptografia na mensagem

Modificar diretamente a classe `Notificacao` pode não ser a melhor solução.

Para resolver esse tipo de situação, podemos utilizar o Decorator Pattern.

4.2. O que é o Decorator Pattern

O Decorator Pattern é um padrão estrutural que permite **adicionar novas funcionalidades a um objeto de forma dinâmica**, sem modificar sua estrutura original.

Em vez de alterar a classe existente, o padrão utiliza **objetos decoradores**, que envolvem o objeto original e acrescentam novos comportamentos.

Essa abordagem permite que novas funcionalidades sejam adicionadas **de forma modular e reutilizável**.

4.3. Ideia do padrão

O Decorator funciona como uma **camada adicional** que envolve um objeto existente.

Podemos imaginar o funcionamento da seguinte forma:

`Objeto original → Decorador → Nova funcionalidade`

É possível aplicar vários decoradores ao mesmo objeto, criando uma **composição de comportamentos**.

4.4. Exemplo em TypeScript

Primeiro definimos uma interface comum.

```
interface Notificacao {
  enviar(mensagem: string): void;
}
```

Implementação básica

```
class NotificacaoBasica implements Notificacao {
  enviar(mensagem: string): void {
    console.log("Enviando mensagem:", mensagem);
  }
}
```

Decorador de log

```
class LogDecorator implements Notificacao {
  constructor(private notificacao: Notificacao) {}

  enviar(mensagem: string): void {
    console.log("Registrando log da mensagem");

    this.notificacao.enviar(mensagem);
  }
}
```

Decorador de criptografia

```
class CriptografiaDecorator implements Notificacao {
  constructor(private notificacao: Notificacao) {}

  enviar(mensagem: string): void {
    const mensagemCriptografada = "***" + mensagem + "***";

    this.notificacao.enviar(mensagemCriptografada);
  }
}
```

Utilizando os decoradores

```
let notificacao: Notificacao = new NotificacaoBasica();
notificacao = new LogDecorator(notificacao);
notificacao = new CriptografiaDecorator(notificacao);
notificacao.enviar("Olá usuário");
```

Saída esperada

```
Registrando log da mensagem
Enviando mensagem: ***Olá usuário***
```

Observe que o objeto original continua o mesmo, mas novos comportamentos foram adicionados.

4.6. Exemplo em aplicações reais

O Decorator aparece com frequência em aplicações modernas.

Middleware em frameworks backend

Em frameworks como Express.js, cada middleware funciona de forma semelhante a um decorador.

```
Request → Middleware → Middleware → Controller
```

Cada camada adiciona um comportamento adicional.

Componentes em bibliotecas de interface

Em bibliotecas de interface, como React, também é comum envolver componentes com funcionalidades adicionais.

Exemplo:

```
Componente → HOC → Novo comportamento
```

HOC significa Higher-Order Component (Componente de Ordem Superior).

Esse é um conceito utilizado principalmente em React, e a ideia é muito semelhante ao Decorator Pattern: um componente recebe outro componente e **adiciona um comportamento extra a ele**, sem modificar o componente original.

Exemplo:

Suponha que temos um componente simples.

```
function Perfil() {
  return <h1>Perfil do usuário</h1>;
}
```

Agora queremos adicionar **verificação de autenticação** antes de renderizar esse componente.

Podemos criar um HOC.

```
function withAuth(Component: React.ComponentType) {
  return function AuthComponent() {
    const autenticado = false; // alterne para ver a diferença

    if (!autenticado) {
      return <h1>Acesso negado</h1>;
    }

    return <Component />;
  };
}
```

Aplicando o HOC

```
const PerfilProtegido = withAuth(Perfil);
```

Fluxo

```
Perfil → withAuth → PerfilProtegido
```

Agora o componente possui **um novo comportamento (verificação de autenticação)**.

```
export default function App() {
    return (
    <div>
      <Perfil />
      <PerfilProtegido />
    </div>
  );
}
```

Exercícios

Veja o vídeo se tiver dúvidas: https://youtu.be/QelBA1UBlpE

Exercício 1 – Facade Pattern

Contexto

Um sistema de reserva de viagens precisa executar várias etapas para concluir uma reserva:

- verificar disponibilidade de voo
- reservar hotel
- alugar carro
- gerar confirmação da viagem

As classes já existem:

```
class VooService {
  verificarDisponibilidade() {
    console.log("Voo disponível");
  }
}

class HotelService {
  reservarHotel() {
    console.log("Hotel reservado");
  }
}

class CarroService {
  alugarCarro() {
    console.log("Carro alugado");
  }
}

class ConfirmacaoService {
  gerarConfirmacao() {
    console.log("Confirmação da viagem enviada");
  }
}
```

Atualmente, qualquer parte do sistema precisa chamar todos esses serviços manualmente.

Parte A

Crie uma classe chamada `ViagemFacade` que simplifique o processo de reserva.

Essa classe deve possuir o método `reservarViagem()`.

Parte B

Utilize a facade para realizar uma reserva de viagem.

Exercício 2 — Adapter Pattern

Contexto

Um sistema de relatórios utiliza a seguinte interface para exportação de dados:

```
interface Exportador {
  exportar(dados: string): void;
}
```

Entretanto, uma biblioteca externa de geração de PDF possui a seguinte interface:

```
class BibliotecaPDF {
  gerarPDF(conteudo: string) {
    console.log("PDF gerado com conteúdo:", conteudo);
```

```
    }
  }
```

O sistema precisa utilizar essa biblioteca sem modificar seu código.

Parte A

Crie uma classe chamada `PDFAdapter` que adapte `BibliotecaPDF` para a interface `Exportador`.

Parte B

Utilize o adapter para exportar um relatório.

Exercício 3 — Decorator Pattern

Contexto

Um sistema possui uma classe responsável por calcular o preço de um produto.

```
interface Produto {
  calcularPreco(): number;
}

class ProdutoBase implements Produto {
  calcularPreco(): number {
    return 100;
  }
}
```

Agora o sistema precisa permitir adicionar taxas ou descontos ao preço do produto.

Exemplos:

- imposto
- desconto promocional

Essas funcionalidades devem ser adicionadas **sem modificar a classe original**.

Parte A

Crie um decorador chamado `ImpostoDecorator` que adiciona 10% de imposto ao preço.

Parte B

Crie um decorador chamado `DescontoDecorator` que aplica 20% de desconto.

Parte C

Combine os decoradores e calcule o preço final.